

从 C++ 到 AI 计算：第二册

Linux 现代 CPU、多核并发、AI 基础、算子开发与量化推理引擎

CPU Performance Study

本册接续第一册的机器执行模型，先建立 Linux 上的现代 CPU 性能工程能力，再进入 AI 模型、算子开发和本地量化推理引擎实现。正文以 LaTeX 正式书稿为准，PDF 和 EPUB 使用同一份源文件生成。

前言

第一册讲清楚了一个 C++ 程序怎样变成进程、指令、寄存器、栈帧、二进制接口和可测量的性能现象。第二册不能直接跳到 AI 算子，因为算子开发不是只会写矩阵乘就够了。真正能在本地 Linux 上跑起来的 CPU 推理引擎，先要理解现代多核心 CPU、缓存层级、TLB、预取、NUMA、线程、锁、原子操作、内存模型、性能计数器和软件优化边界。没有这些底座，后面的矩阵乘、量化、Attention 和推理图都只会变成孤立技巧。

本册的路径因此重新定位为三步。第一步是现代 CPU 性能工程：在 Linux 上认识多核机器，知道数据怎样穿过 cache line、TLB、内存带宽和执行单元，知道 benchmark 怎样写才可信。第二步是并发和软件优化：理解线程池、任务切分、锁、原子、false sharing、内存序和缓存友好布局，能解释为什么加线程有时更慢。第三步才进入 AI：从张量、模型、推理图、算子、参数、激活和量化格式讲起，让没有 AI 背景的读者也能逐步读懂模型推理，然后开始实现 CPU 算子和本地推理引擎。

本册有一个贯穿全书的大作业：实现一个 Linux 本地 CPU 版本的量化推理引擎。这个项目不是最后一章临时出现，而是从第一章开始逐步搭建。前面章节先建立工程边界和测量框架，中间章节补线程池、张量结构、内存布局和基础算子，后面章节加入量化权重、模型加载、推理图执行、端到端 benchmark 和正确性验证。最终目标是在本地 Linux 机器上，不依赖云服务，加载一个小型量化模型并完成推理。

核心思想

第二册的核心不是“AI 很快”，而是“怎样在 Linux 上把现代 CPU、多核并发、内存层级和 AI 算子组织成一个能运行、能验证、能测量、能继续优化的量化推理系统”。

本册仍然保持第一册的写法：原理先行，代码作为证据；实验必须说明输入、环境、命令和误差；源码阅读抓数据结构、热路径和性能边界。读者读完后，不应该只会背 SIMD、GEMM、Attention 这些名词，而应该能从零解释一个本地 CPU 推理引擎需要哪些模块，能写出可测试的 C++ 组件，能用 Linux 工具定位瓶颈，并能把量化模型跑通。

怎样阅读本册

本册默认目标平台是本地 Linux 机器。读者可以使用普通 x86-64 Linux 台式机、笔记本或服务器完成完整实验；Termux 里可用的 Linux-like 环境适合阅读、构建小模型和运行基础测试，但不作为本册性能结论的标准平台。涉及 perf、NUMA、线程亲和性和 CPU 计数器时，不同环境能力会有差异，实验报告必须写清楚限制。

阅读时不要跳过前半本。AI 算子开发的难点不只是公式，而是公式如何落在真实硬件和软件运行时上。缓存 miss、TLB miss、false sharing、锁竞争、内存带宽饱和、线程切分过碎、数据布局不匹配，都会让看似正确的算子跑不快。先读现代 CPU 和并发部分，是为了让后面的 AI 内容有工程根基。

本册的贯穿项目叫 Linux CPU Quantized Inference。每读完一部分，就给项目增加一个可运行模块。第一部分完成工程骨架、张量容器和 benchmark 框架；第二部分加入线程池、同步原语和 SIMD 友好的循环；第三部分补 AI 基础，理解模型、张量、权重、激活、推理图和常见算子；第四部分实现 GEMM、归一化、Softmax 等 CPU 内核；第五部分加入 int8 量化权重、模型文件、推理执行器和端到端测试。

心智模型

这本书的阅读顺序就是项目的成长顺序：先知道机器，后组织并发，再理解 AI 模型，然后写算子，最后把算子接成推理引擎。

每章阅读时建议做三件事。第一，画出本章机制的边界，例如 cache line 管什么、锁保护什么、张量 stride 表达什么。第二，把机制放进贯穿项目，问它会影响哪个模块。第三，写一个最小实验验证它，例如改变线程数、改变矩阵布局、改变量化 scale 或改变 batch size。只读文字不跑实验，很难真正掌握性能工程。

常见误区

不要把 AI 部分理解成“调用现成框架”。本册要训练的是算子开发和推理引擎实现能力。即使用小模型、小算子和教学格式，也必须保持真实工程的基本约束：输入可验证、模型可加载、结果可检查、性能可测量。

全书结构

本册分为五个部分，围绕同一个大作业展开：在 Linux 本地实现一个 CPU 量化推理引擎。

第一部分是 Linux 上的现代 CPU 性能工程。它先定义贯穿项目和本地 Linux 环境，再解释现代多核心 CPU、缓存、TLB、预取、内存带宽、NUMA、性能模型和可信测量。读者在这里要获得一个判断能力：程序慢，到底更像计算问题、内存问题、同步问题还是测量问题。

第二部分是并发、同步与软件优化。它讲线程、任务切分、锁、原子操作、内存模型、false sharing、SIMD 和缓存友好布局。推理引擎最终一定要处理多线程执行，因此这一部分要先把运行时基础讲清楚，再谈 AI 算子。

第三部分是 AI 基础。它从零解释张量、shape、stride、dtype、参数、激活、模型、推理图、算子、Embedding、MLP、CNN、Transformer 和 Attention。这个部分的目标不是训练模型，而是让没有 AI 背景的读者能看懂推理时到底在算什么、数据怎样流动、算子接口应该怎样设计。

第四部分是算子开发。它把数学公式落到 CPU 内核上：矩阵乘从三重循环到分块和 packing，卷积和归一化从数据流角度分析，Softmax 解释数值稳定性和归约结构。所有算子都服务于贯穿项目，而不是孤立示例。

第五部分是量化推理引擎。它把 Attention、KV Cache、int8 量化、低精度累加、模型文件、推理图执行、benchmark suite 和源码阅读连接起来。最终项目要能在 Linux 本地加载一个教学量化模型，执行推理，输出可验证结果，并报告延迟、吞吐和主要瓶颈。

核心思想

全书的结构不是“CPU 知识加 AI 知识”的拼接，而是一条工程链：Linux 机器模型决定运行边界，并发运行时决定多核执行方式，AI 基础决定算子接口，算子内核决定局部性能，量化推理引擎把所有部件接成系统。

目录

第一部分	Linux 上的现代 CPU 性能工程	
第 1 章	Linux 本地 CPU 推理引擎项目	2
第 2 章	现代多核心 CPU 的执行结构	4
第 3 章	缓存、TLB、预取与数据布局	6
第 4 章	Roofline 模型与可信测量	8
第二部分	并发、同步与软件优化	
第 5 章	线程、锁、原子操作与内存模型	11
第 6 章	SIMD：从标量循环到向量化	14
第 7 章	缓存友好的软件优化	16
第三部分	AI 基础：从张量到推理图	
第 8 章	AI 模型、张量与推理图	19
第 9 章	常见 AI 算子与模型结构	21
第四部分	算子开发：从数学到 CPU 内核	
第 10 章	矩阵乘：从三重循环到性能结构	24
第 11 章	分块、Packing 与微内核	27
第 12 章	卷积、归一化与 Softmax 算子	30
第五部分	量化推理引擎：从算子到本地运行	
第 13 章	Attention、KV Cache 与内存流量	33
第 14 章	本地量化推理引擎	35
附录 A	实验路线	37
附录 B	源码阅读索引	38

第零部分

Linux 上的现代 CPU 性能工程

第 1 章

Linux 本地 CPU 推理引擎项目

第二册从一个明确的大作业开始：在 Linux 本地实现一个 CPU 版本的量化推理引擎。这个项目不依赖云服务，不调用现成深度学习框架完成核心推理，不把算子实现藏在黑盒库里。它的目标不是一开始就追求工业级完整度，而是让读者亲手打通推理系统的骨架：模型文件、张量、量化权重、算子、线程运行时、推理图、正确性测试和性能报告。

1.1 为什么先定义项目

如果直接从 AI 名词讲起，读者很容易分不清哪些概念服务于训练，哪些概念服务于推理，哪些概念只是框架 API。推理引擎给我们一个清晰边界：输入是一段数据和一个模型文件，输出是模型的预测结果；中间过程包括加载参数、组织张量、执行算子、管理内存、调度线程和记录性能。只要这条路径跑通，后面的矩阵乘、量化、Softmax、Attention 才有落脚点。

本册的教学项目命名为 Linux CPU Quantized Inference，简称 LCQI。它先实现一个量化 MLP 模型，再逐步扩展到更复杂的算子。选择 MLP 作为第一阶段，是因为它包含推理引擎最基本的部件：输入张量、int8 权重、scale、矩阵向量乘、bias、ReLU 和输出分类。模型小，但路径真实。

核心思想

贯穿项目的价值不在于模型大，而在于链路完整。一个小模型只要能加载、能推理、能验证、能测量，就已经具备继续扩展到卷积、Attention 和 Transformer 的工程骨架。

1.2 项目边界

LCQI 的第一阶段只支持本地 Linux 机器上的 CPU 执行，C++20 编写。这里的“本地”不是云端服务，也不是把推理委托给深度学习框架；这里的“Linux”是实验和性能分析的标准平台。模型格式使用教学文本文件，避免读者一开始被复杂二进制格式拖住。权重使用 int8 存储，每一层有一个 scale，把 int8 权重反量化到 float 参与计算。输入和激活先用 float，后续章节再讨论激活量化和更高效的 int8 累加。

第一阶段模型结构固定为两层 MLP：

- 输入向量长度为 4。
- 第一层输出隐藏向量，执行量化权重矩阵乘、bias 和 ReLU。
- 第二层输出分类 logits。
- 推理结果取最大 logit 的下标。

这个结构足够小，可以手工检查结果；也足够真实，因为大模型中的 MLP、投影层和分类头都可以看作更大规模的矩阵运算。

1.3 工程模块

LCQI 至少包含七个模块。第一是模型加载器，把本地模型文件解析成内存中的权重和 bias。第二是张量结构，表达 shape、元素数量和连续数据。第三是量化权重结构，保存 int8 数据、scale、输入维度和输出维度。第四是算子内核，先实现量化线性层和 ReLU。第五是推理执行器，把多层算子按顺序执行。第六是测试，用固定模型和输入验证输出。第七是 benchmark，记录延迟和吞吐。

这些模块会随着章节推进不断变强。讲缓存时，我们检查张量连续性；讲线程时，我们给线性层增加并行输出切分；讲 SIMD 时，我们分析内层点积；讲 AI 基础时，我们把线性层、激活和 logits 放进模型语义；讲量化时，我们改进权重格式和误差检查。

Listing 1.1 推理引擎第一阶段的核心对象关系


```

1 struct QuantizedLinearLayer {
2     std::int32_t input_size;
3     std::int32_t output_size;
4     float scale;
5     std::vector<std::int8_t> weights;
6     std::vector<float> bias;
7 };
8
9 struct TinyMlpModel {
10     QuantizedLinearLayer hidden;
11     QuantizedLinearLayer output;
12 };

```

这段结构不是最终设计，但已经表达了推理引擎的基本事实：模型不是魔法对象，而是一组带形状、类型和量化元数据的数组。算子开发的第一步，就是让这些数组的含义清楚。

1.4 Linux 环境要求

本册实验以 Linux 为目标环境。最低要求是能使用 CMake、支持 C++20 的编译器和基本命令行工具。推荐工具包括 `cmake`、`ctest`、`perf`、`taskset`、`lscpu`、`numactl`。手机 Termux 环境可以完成大部分 C++ 构建和小模型推理，但 `perf`、NUMA 和硬件计数器可能受权限或内核限制。

实验报告必须记录系统信息。至少包括 CPU 型号、核心数、线程数、内存大小、编译器版本、编译选项、操作系统版本和是否运行在容器或手机环境中。性能工程的第一条纪律是：没有环境上下文的数字不能比较。

本章实验：构建 LCQI 项目，运行内置测试，确认模型文件能被加载，推理输出能通过断言。随后记录一次本地环境报告，包括 `uname-a`、`lscpu` 和编译器版本。如果某些命令在 Termux 中不可用，要在报告中写明。

1.5 从本章开始积累证据

后续每章都要回到这个项目。现代 CPU 结构解释为什么内存布局重要；并发章节解释为什么线程池不能随便写；AI 基础章节解释模型文件中的权重和算子语义；算子章节解释矩阵乘和归一化如何实现；量化章节解释 `scale`、误差和低精度累加；最后的 benchmark suite 把所有证据汇总成工程报告。

这条路线会慢一点，但它避免了一个常见问题：懂一点 AI 名词，懂一点 SIMD 指令，却不知道怎样把它们接成能运行的系统。本册的目标是把这条链接起来。

第 2 章

现代多核心 CPU 的执行结构

现代 CPU 不是“一个更快的核心”那么简单。它包含多个核心、每个核心内部的流水线和执行端口、多级缓存、共享末级缓存、内存控制器、硬件预取器、TLB、互连结构和可能存在的 NUMA 节点。推理引擎运行在这样的机器上，性能取决于所有这些结构如何共同工作。

2.1 核心、线程和拓扑

一个物理核心通常包含前端取指和解码、乱序执行窗口、整数和浮点执行单元、加载和存储单元、寄存器文件以及私有 L1/L2 缓存。超线程或 SMT 让一个物理核心暴露为多个逻辑 CPU，多个硬件线程共享部分核心资源。对计算密集型内核而言，SMT 不一定带来线性加速；对等待内存的内核而言，SMT 有时能隐藏一部分延迟。

Linux 把可调度对象放到逻辑 CPU 上运行。读者在 benchmark 时看到的“线程数”，不等于一定占用了同样数量的物理核心。若线程在不同核心之间迁移，缓存热度会被破坏；若多个重计算线程落在同一物理核心的两个 SMT 线程上，浮点执行资源会竞争。因此推理引擎 benchmark 要记录核心拓扑，并在必要时使用线程亲和性。

心智模型

多核心 CPU 是一组共享内存的执行工人。每个工人有自己的近身工具箱，也共享仓库和道路。任务切分好不好，不只看工人数，还要看工具箱是否够用、道路是否拥堵、工人是否反复换位置。

2.2 流水线和乱序执行

CPU 不会逐行机械执行 C++ 代码。前端把指令取出、解码成内部微操作，后端在依赖允许时乱序调度到不同执行端口。若两条指令没有数据依赖，硬件可以并行推进；若一条指令依赖上一条结果，就形成依赖链。点积中的单累加器就是典型依赖链，多个累加器能让硬件看到更多独立工作。

推理引擎中的算子常常有大量重复循环。循环体是否容易解码，地址计算是否复杂，加载是否及时返回，FMA 是否有足够独立输入，都会影响单核吞吐。编译器优化和手写内核的目的，不是让代码看起来短，而是让硬件后端有连续、规则、独立的工作可以调度。

2.3 共享缓存和一致性

每个核心通常有私有 L1 数据缓存和 L2 缓存，多个核心共享 L3 或 LLC。核心之间通过缓存一致性协议维护同一内存地址的可见性。读共享数据通常比较便宜，多个核心同时写相邻数据就可能触发 cache line 来回转移。false sharing 正是这种现象：逻辑上不同变量，物理上落在同一 cache line，导致一致性流量。

推理引擎中，权重通常是只读共享数据，适合被多个线程读取；输出张量应该按块分配给不同线程写，避免写同一 cache line；线程本地统计量要考虑对齐和填充。多线程性能问题往往不是“锁慢”这么简单，而是共享写、同步点和缓存一致性共同作用。

2.4 内存控制器和 NUMA

单 socket 机器通常有一个或多个内存通道，多 socket 机器可能有 NUMA 节点。线程访问本地节点内存和远端节点内存的延迟、带宽不同。大模型权重和 KV Cache 占用很大，NUMA 放置会影响吞吐和尾延迟。即使第一阶段 LCQI 模型很小，本册也要提前建立 NUMA 意识，因为后续扩展到大模型时这个问题无法回避。

常见误区

不要把“核心数”当成唯一性能指标。同样 16 个逻辑 CPU，可能是 8 个物理核心加 SMT，也可能是大小核混合，也可能跨 NUMA 节点。性能报告必须写清楚拓扑，否则线程扩展结果无法解释。

2.5 和推理引擎的关系

LCQI 的每个模块都会受到 CPU 结构影响。模型加载主要受 IO 和解析影响；张量遍历受缓存和 TLB 影响；量化线性层受内存带宽、SIMD 和依赖链影响；线程池受调度、同步和缓存一致性影响；benchmark 受频率、亲和性和系统噪声影响。理解 CPU 结构，是为了把这些现象放回正确层级。

习题与作业

在本地 Linux 上运行 `lscpu`，记录 socket、core、thread、cache 和 NUMA 信息。写一段短报告解释：如果 LCQI 的线性层使用 1、2、4、8 个线程，你预计最先遇到的瓶颈可能是什么。

第 3 章

缓存、TLB、预取与数据布局

CPU 执行单元的速度很快，但它只能处理已经到达寄存器的数据。数据从内存到寄存器之间要经过多级缓存、TLB、加载队列、存储队列和预取器。AI 算子的大部分优化都在处理同一个矛盾：数学上相邻的元素，不一定在内存里相邻；代码里重复使用的值，不一定能在缓存里留到下一次使用。

3.1 缓存层级不是抽象背景

缓存以 cache line 为基本单位搬运数据，常见大小是 64 字节。读取一个 float 时，硬件通常会把同一条 cache line 里的相邻元素一起带进来。如果后续代码顺序访问这些元素，带宽利用率高；如果每次跨很大步长访问，只用到每条 cache line 里的一个元素，实际带宽会被浪费。

矩阵按行主序存储时，连续访问一行是顺序访问，连续访问一列是跨步访问。第一册已经讲过地址计算，第二册要进一步问：这个地址序列对 cache line 是否友好，对硬件预取器是否友好，对 TLB 是否友好。一个三重循环的数学结果不变，但循环顺序不同，内存访问轨迹完全不同。

Listing 3.1 同一个矩阵，行访问和列访问的内存轨迹不同

```
1 #include <cstdint>
2 #include <cstdint>
3 #include <span>
4
5 float sum_by_rows(std::span<const float> a,
6                  std::int32_t rows,
7                  std::int32_t cols) {
8     float acc = 0.0F;
9     for (std::int32_t r = 0; r < rows; ++r) {
10         for (std::int32_t c = 0; c < cols; ++c) {
11             acc += a[static_cast<std::size_t>(r) * cols + c];
12         }
13     }
14     return acc;
15 }
16
17 float sum_by_cols(std::span<const float> a,
18                  std::int32_t rows,
19                  std::int32_t cols) {
20     float acc = 0.0F;
21     for (std::int32_t c = 0; c < cols; ++c) {
22         for (std::int32_t r = 0; r < rows; ++r) {
23             acc += a[static_cast<std::size_t>(r) * cols + c];
24         }
25     }
26     return acc;
27 }
```

当矩阵很小，两个函数都在缓存里跑，差距可能不明显；当矩阵超过缓存容量，列访问就会暴露跨步访问的问题。AI 计算中的很多布局变换，例如 NHWC 和 NCHW 的选择、权重预打包、KV Cache 的排布，最终都要回到这个问题：下一次访问的数据，是否已经在离核心足够近的层级里。

3.2 TLB 和页粒度

缓存按 cache line 管数据，TLB 按页缓存虚拟地址到物理地址的翻译。访问大量离散页时，即使数据总量不算夸张，也可能因为 TLB miss 变慢。大模型推理中，权重很大，KV Cache 随序列增长，内存访问不仅要考虑缓存容量，还要考虑页表翻译和 NUMA 放置。

TLB 问题常被初学者忽略，因为代码里看不到它。一个指针加下标的表达式背后，硬件要完成地址生成、权限检查和地址翻译。若访问模式顺序，TLB 和预取器都容易工作；若访问模式随机，硬件很难提前准备。Embedding lookup、稀疏访问、哈希结构和不规则图计算通常比规则矩阵乘更容易受 TLB 和缓存 miss 影响。

3.3 AoS、SoA 与结构化数据

AoS (Array of Structures) 把一个对象的多个字段放在一起，适合按对象访问；**SoA** (Structure of Arrays) 把每个字段分别放成数组，适合 SIMD 和批量处理。AI 算子通常处理大批同类型数值，因此 SoA 或接近 SoA 的布局更常见。系统代码中也会遇到两者取舍：一个推理请求对象可能适合 AoS，但一批 token 的 logits 或概率更适合连续数组。

Listing 3.2 AoS 和 SoA 的访问形状

```
1 #include <stdint>
2 #include <vector>
3
4 struct TokenStateAos {
5     float logit;
6     float probability;
7     std::int32_t token_id;
8 };
9
10 struct TokenStateSoa {
11     std::vector<float> logits;
12     std::vector<float> probabilities;
13     std::vector<std::int32_t> token_ids;
14 };
```

如果代码要对所有 logit 做 Softmax，SoA 更容易顺序扫描，也更容易向量化。如果代码经常拿某一个 token 的全部字段做复杂逻辑，AoS 可能更自然。布局没有绝对好坏，只有访问模式是否匹配。

3.4 预取与局部性边界

硬件预取器擅长发现顺序和固定步长访问，不擅长预测复杂间接访问。手动预取有时有用，但必须谨慎。预取太晚，数据仍然来不及；预取太早，数据可能在使用前被挤出缓存；预取太多，会污染缓存并占用带宽。本册会优先强调通过布局和循环顺序创造自然局部性，而不是一开始就依赖手动预取。

核心思想

缓存优化不是把所有数据都塞进缓存，而是让即将重复使用的数据在重复使用前不要离开合适的缓存层级。分块、packing、循环交换和算子融合都是围绕这个目标展开。

习题与作业

把一个大矩阵分别按行和按列求和，记录不同矩阵规模下的运行时间。矩阵从 L1 cache 能容纳的规模开始，逐步扩大到 L2、L3 之外。观察差距从哪里开始变大，并结合 cache line 解释原因。

第 4 章

Roofline 模型与可信测量

性能优化最危险的习惯，是看到时间变短就认为自己理解了原因。时间只是结果，不是解释。一个程序变快，可能是指令少了，可能是缓存命中好了，可能是编译器做了新的向量化，也可能只是输入刚好留在缓存里。Roofline 模型提供了一个粗但非常有用的坐标系：横轴看计算强度，也就是每搬一个字节做多少操作；纵轴看实际吞吐；硬件的计算峰值和内存带宽给出上界。

4.1 计算强度的意义

计算强度 (arithmetic intensity) 定义为操作数除以字节数。若一个算子做了很多乘加，但只需要少量数据并能反复复用，那么计算强度高；若一个算子只是读一个元素、做很少计算、写一个元素，那么计算强度低。高计算强度算子更可能被浮点吞吐限制，低计算强度算子更可能被内存带宽限制。

例如向量加法 $C = A + B$ ，每个元素读两个 float、写一个 float，大约移动十二个字节，只做一次加法，计算强度很低。矩阵乘 $C = AB$ 如果分块做得好，每个块里的元素会被重复使用，计算强度可以很高。Softmax 介于两者之间：它有指数、求和和除法，看起来计算不少，但通常需要多次扫描同一行，还涉及数值稳定性和归约，最终瓶颈取决于行长度、实现方式和向量化程度。

深入理解

Roofline 不是精确模拟器。它不会告诉你分支预测、端口压力、TLB miss、依赖链、前端解码和线程同步的全部细节。它的价值在于先排除不可能：如果计算强度很低，而你的吞吐已经接近内存带宽上界，再继续手写 FMA 通常不会带来数量级提升。

4.2 建立测量边界

可信测量要先定义边界。测单个内核时，输入数据是否包含初始化时间，是否包含内存分配，是否包含线程池创建，是否包含随机数生成，都必须说清楚。测端到端推理时，也要区分 prefill、decode、采样、IO、tokenization 和模型计算。边界不同，数字没有可比性。

最小的 benchmark 应该至少记录五件事：机器和编译器信息、输入形状、重复次数、预热策略、计时范围。预热的目的不是“让数字好看”，而是让动态链接、页错误、缓存冷启动、频率变化等一次性成本不污染稳定阶段。重复次数不能机械固定，应当让总运行时间足够长，避免计时器分辨率和调度抖动主导结果。

Listing 4.1 最小计时框架：只计核心循环

```
1 #include <chrono>
2 #include <cstdint>
3 #include <functional>
4 #include <iostream>
5
6 double measure_seconds(const std::function<void()>& work,
7                        std::int32_t warmup,
8                        std::int32_t repeat) {
9     for (std::int32_t i = 0; i < warmup; ++i) {
10         work();
11     }
12
13     const auto begin = std::chrono::steady_clock::now();
14     for (std::int32_t i = 0; i < repeat; ++i) {
```



```

15     work();
16 }
17 const auto end = std::chrono::steady_clock::now();
18 const std::chrono::duration<double> elapsed = end - begin;
19 return elapsed.count() / static_cast<double>(repeat);
20 }

```

这个框架仍然很朴素。真实项目中还要防止编译器把结果优化掉，要固定输入或记录随机种子，要把输出校验和写到不可优化的位置，要记录 CPU 频率策略。若测多线程，还要说明线程数、亲和性、NUMA 节点和是否与系统其他任务竞争。

4.3 计数器证据

时间告诉我们结果，性能计数器帮助解释原因。在 Linux 上常用 perf stat、perf record 和 perf report。对于本册的主题，常看的指标可以分成三类：执行侧看 cycles、instructions、branches 和 branch-misses；缓存侧看 cache-references、cache-misses、L1 data cache load misses、LLC misses 和 dTLB misses；平台相关部分再补充 SIMD 浮点计数器和内存带宽事件。

计数器也有误差。不同 CPU 的事件名不同，虚拟化环境可能屏蔽部分事件，过短程序的采样不稳定，多线程程序的进程级计数可能混入调度影响。因此计数器不能机械崇拜，而要与模型对照。若 Roofline 估算认为程序应受内存限制，而 perf 显示 LLC miss 和带宽很高，这两者互相支持；若计数器与模型冲突，就要检查输入规模、缓存热度、编译选项和测量边界。

实验建议：写三个内核，分别是向量加法、点积和朴素矩阵乘。对每个内核记录运行时间、估算 FLOPs、估算读写字节，并用 **perfstat** 查看 cycles、instructions 和 cache-misses。观察哪个内核更接近内存带宽问题，哪个内核更接近计算吞吐问题。

4.4 从数字回到决策

测量的最终目的不是报一个漂亮数字，而是做决策。若一个矩阵乘只有理论峰值的百分之五，且 cache miss 很高，那么下一步可能是改循环顺序、分块或 packing；若一个逐元素激活已经达到很高内存带宽，那么下一步可能是算子融合，减少中间张量读写；若多线程从四核到八核不再加速，那么下一步要看带宽饱和、任务粒度、同步和 NUMA。

这就是本册后续章节的工作方式。每个技术点都要回到三个问题：它改变了计算账本还是内存账本，它把瓶颈从哪里移动到哪里，它能被什么实验验证。无法回答这三个问题的优化，最多算一次尝试，不能算工程结论。

常见误区

不要只报告最快一次。最快一次通常最不代表真实服务，因为它避开了调度抖动、缓存冷启动和尾延迟。教材中的内核实验可以看平均值，系统实验还要看分位数和稳定性。

第零部分

并发、同步与软件优化

第 5 章

线程、锁、原子操作与内存模型

推理引擎要使用多核，就必须处理并发。并发不是简单地把循环丢给多个线程。线程之间会共享模型、共享输入、写输出、更新队列、等待任务完成。只要存在共享状态，就会遇到锁、原子操作、内存可见性和数据竞争问题。性能工程里，错误的并发代码比慢代码更危险，因为它可能偶发出错，并且很难复现。

5.1 线程解决什么问题

线程让多个执行流在同一进程地址空间内运行。对推理引擎而言，线程常用于并行计算输出通道、并行处理 batch、并行处理 attention head 或并行执行独立算子。线程共享模型权重可以避免复制大对象，但共享也意味着必须明确写入所有权。

第一阶段 LCQI 的线性层可以按输出神经元切分。每个线程负责一段输出下标，读取同一份输入和权重，写不同输出位置。这种切分简单、安全，适合教学。按输入维度切分则需要多个线程写同一个输出的部分和，最后再归约，复杂度更高。

5.2 锁和临界区

锁保护临界区，保证同一时刻只有一个线程修改共享状态。任务队列、日志、全局统计、内存池元数据都可能需要锁。锁的成本不只在加锁和解锁本身，还包括线程阻塞、唤醒、缓存行争用和优先级反转。临界区越小越好，共享写越少越好。

Listing 5.1 用互斥锁保护共享计数器

```
1 #include <stdint>
2 #include <mutex>
3
4 class SafeCounter {
5 public:
6     void add(std::int64_t value) {
7         std::lock_guard<std::mutex> guard(this->mutex_);
8         this->value_ += value;
9     }
10
11     std::int64_t value() const {
12         std::lock_guard<std::mutex> guard(this->mutex_);
13         return this->value_;
14     }
15
16 private:
17     mutable std::mutex mutex_;
18     std::int64_t value_ = 0;
19 };
```

这个例子使用了明确的 `this->` 成员访问，适合小范围共享状态。真正的热路径不应频繁进入这种临界区。如果每个输出元素都加锁一次，推理引擎会被同步成本拖垮。正确做法通常是线程本地累计，最后合并。

5.3 原子操作和内存序

原子操作可以在不使用互斥锁的情况下安全读写单个共享变量。它适合计数器、状态标志、任务索引等简单场景。原子操作仍然有成本，并且内存序会影响可见性和重排。默认的 sequentially consistent 最容易理解，但可能比必要约束更强；relaxed 适合只需要原子性、不需要同步其他内存的计数器。

Listing 5.2 任务索引用原子递增分发

```

1 #include <atomic>
2 #include <cstdint>
3
4 class AtomicTaskCursor {
5 public:
6     explicit AtomicTaskCursor(std::int32_t end) : end_(end) {}
7
8     bool next(std::int32_t* task) {
9         const std::int32_t current =
10             this->cursor_.fetch_add(1, std::memory_order_relaxed);
11         if (current >= this->end_) {
12             return false;
13         }
14         *task = current;
15         return true;
16     }
17
18 private:
19     std::atomic<std::int32_t> cursor_{0};
20     std::int32_t end_ = 0;
21 };

```

这里使用 relaxed 是因为任务编号本身不发布其他数据，所有任务数据在启动线程前已经准备好。若一个线程写入数据并通过原子标志通知另一个线程读取，就需要 release/acquire 语义。内存模型的核心问题是：一个线程写的内容，另一个线程何时保证看见。

5.4 线程池和任务切分

推理引擎不能每次算子调用都创建线程。线程池提前创建工作线程，算子提交任务，线程池执行并等待完成。线程池设计要控制任务粒度，避免过度切分；要避免所有线程抢同一个锁；要明确异常和停止机制。第一阶段可以实现简单 parallel for，后续再引入 work stealing 或 NUMA 感知。

核心思想

并发优化的第一原则是写入所有权清晰。让每个线程写独立输出块，通常比让多个线程共同更新同一结果更容易正确，也更容易获得性能。

5.5 推理引擎里的并发边界

LCQI 中，模型权重加载后只读，可以在线程间共享；输入张量一次推理内只读；输出张量按区间分配；任务队列和 benchmark 统计需要同步；日志不应出现在热路径。这个边界要在代码接口里体现出来，否则后续算子越多，数据竞争越难排查。

习题与作业

把 LCQI 的隐藏层输出按区间切给多个线程。要求每个线程只写自己的输出区间，主线程等待所有任务完成后再进入下一层。比较 1、2、4 个线程的时间，并解释加速或不加

速的原因。

第 6 章

SIMD：从标量循环到向量化

SIMD 的全称是 Single Instruction Multiple Data，即一条指令处理多个数据 lane。对 AI 计算而言，SIMD 不是装饰，而是单核吞吐的核心来源。一个标量循环每次处理一个 float，向量化后一次可以处理四个、八个、十六个甚至更多元素。问题在于，数学上能并行，不代表代码一定能被编译器或手写 intrinsic 正确高效地向量化。

6.1 向量化需要独立性

最容易向量化的是逐元素循环。例如 $C_i = A_i + B_i$ ，每个元素的计算互不依赖，地址连续，循环次数清楚。编译器可以把多个元素合成一个向量加载、一个向量加法和一个向量存储。若循环中存在跨迭代依赖，例如前缀和，后一个元素依赖前一个元素的结果，向量化就困难得多。

Listing 6.1 容易向量化的逐元素循环

```
1 #include <cstdint>
2 #include <span>
3
4 void add_arrays(std::span<const float> a,
5               std::span<const float> b,
6               std::span<float> c) {
7     const std::int64_t n = static_cast<std::int64_t>(c.size());
8     for (std::int64_t i = 0; i < n; ++i) {
9         c[static_cast<std::size_t>(i)] =
10            a[static_cast<std::size_t>(i)] + b[static_cast<std::size_t>(i)];
11     }
12 }
```

这段代码看起来普通，但它具备向量化的基本条件：连续内存、固定步长、没有循环携带依赖。若把输出写到输入重叠的区域，或者通过不透明指针访问，编译器就可能因为别名风险保守。工业代码会通过接口约束、对齐信息、restrict 风格提示或库内部封装减少这种不确定性。

6.2 归约为什么更复杂

点积也是 AI 计算常见循环，但它比逐元素加法复杂，因为所有迭代都更新同一个累加器。向量化点积通常不是让多个 lane 同时写一个标量，而是维护多个部分和，最后再做水平归约。这样做可以减少依赖链长度，提高指令级并行。

Listing 6.2 教学版点积：多个累加器降低依赖链压力

```
1 #include <cstdint>
2 #include <span>
3
4 float dot_unrolled(std::span<const float> a,
5                   std::span<const float> b) {
6     const std::int64_t n = static_cast<std::int64_t>(a.size());
7     float acc0 = 0.0F;
8     float acc1 = 0.0F;
9     float acc2 = 0.0F;
10    float acc3 = 0.0F;
```

```

11
12     std::int64_t i = 0;
13     for (; i + 3 < n; i += 4) {
14         acc0 += a[static_cast<std::size_t>(i)] *
15                b[static_cast<std::size_t>(i)];
16         acc1 += a[static_cast<std::size_t>(i + 1)] *
17                b[static_cast<std::size_t>(i + 1)];
18         acc2 += a[static_cast<std::size_t>(i + 2)] *
19                b[static_cast<std::size_t>(i + 2)];
20         acc3 += a[static_cast<std::size_t>(i + 3)] *
21                b[static_cast<std::size_t>(i + 3)];
22     }
23
24     float acc = (acc0 + acc1) + (acc2 + acc3);
25     for (; i < n; ++i) {
26         acc += a[static_cast<std::size_t>(i)] *
27                b[static_cast<std::size_t>(i)];
28     }
29     return acc;
30 }

```

这不是最终的 SIMD intrinsic 版本，但它揭示了一个重要思想：归约的瓶颈不只是操作数量，还有依赖链。现代 CPU 可以同时执行多条独立浮点指令，但如果每条指令都依赖上一条累加结果，吞吐就会被延迟限制。多个累加器让硬件有更多独立工作可做。

6.3 对齐、尾部和掩码

SIMD 循环通常要处理三个细节。第一是对齐。对齐访问更容易让硬件高效工作，虽然现代 CPU 对非对齐访问的支持已经比过去好很多。第二是尾部。数组长度不一定刚好是向量宽度的整数倍，尾部元素要用标量循环、掩码加载或特殊路径处理。第三是异常和边界。若向量加载越过合法内存，即使多出来的 lane 不参与结果，也可能触发错误；因此边界处理必须明确。

AVX-512 提供更强的掩码机制，能让尾部处理更自然；AVX2 常见做法是主循环处理完整向量，尾部用标量补齐。不同平台有不同权衡。本册不会把所有 intrinsic 当成必背 API，而会先建立向量语义：一组 lane 同步执行，同一条指令处理多个元素，水平归约和跨 lane shuffle 是额外成本。

6.4 编译器自动向量化与手写内核

能自动向量化时，应先让编译器做。编译器知道目标架构、寄存器分配和调度细节，也能随编译选项适配不同平台。但矩阵乘、卷积和量化 GEMM 这类核心内核往往需要手写或生成，因为它们涉及寄存器块、数据 packing、预取、指令选择和微架构调度。手写内核不是为了炫技，而是因为通用编译器很难从普通三重循环推导出工业库需要的全部结构。

常见误区

不要把 SIMD 等同于“把循环步长改成向量宽度”。真正的问题包括别名、对齐、尾部、依赖链、数据布局、寄存器压力和内存带宽。向量化增加了单核计算能力，也可能把瓶颈更快推到内存系统。

实验建议：使用编译器的向量化报告，例如 GCC 的 `-fopt-info-vec` 或 Clang 的 `-Rpass=loop-vectorize`，观察逐元素加法、点积、前缀和三个循环分别能否自动向量化。把报告和汇编对照起来，确认是否出现向量加载和向量浮点指令。

第 7 章

缓存友好的软件优化

软件优化不是看到循环就展开，看到数组就 SIMD。现代 CPU 的大部分性能问题来自数据形状和执行形状不匹配：数据不连续、跨步太大、复用太晚、写入互相干扰、分支破坏预测、临时对象制造额外流量。缓存友好的优化从数据布局开始，而不是从指令开始。

7.1 先减少无意义的移动

推理引擎中，权重、激活、中间张量和输出都会占用内存带宽。若每个算子都把完整中间结果写回内存，再由下一个算子读出，带宽会被快速耗尽。算子融合、原地操作、复用缓冲区和避免临时拷贝，都是减少数据移动的手段。

LCQI 第一阶段的两层 MLP 可以复用中间缓冲区。隐藏层输出需要保存给输出层，但 ReLU 可以在隐藏层写出时直接完成，不需要单独再扫一遍。这个优化很小，却代表了算子融合的基本思想：生产数据时顺手完成简单后处理。

7.2 布局决定访问轨迹

矩阵按行主序存储时，连续访问一行友好，连续访问一列不友好。量化线性层如果每个输出神经元对应一行权重，那么计算一个输出时可以顺序扫描这一行，与输入向量做点积。若后续要并行多个输出神经元，每个线程读取同一输入，写不同输出，权重访问也保持顺序。

Listing 7.1 行主序权重点积的访问形状

```
1 float dot_row_major(const std::int8_t* weights,
2                     const float* input,
3                     float scale,
4                     std::int32_t input_size) {
5     float acc = 0.0F;
6     for (std::int32_t i = 0; i < input_size; ++i) {
7         acc += static_cast<float>(weights[i]) * scale * input[i];
8     }
9     return acc;
10 }
```

这个循环的权重和输入都是顺序访问，适合缓存和预取器。后续要优化时，可以把反量化乘 scale 移到更合适的位置，可以使用多个累加器，也可以在支持的硬件上使用低精度点积指令。但基础访问形状要先正确。

7.3 分支和边界

热循环里的分支会影响预测和向量化。边界检查、padding、mask、不同 dtype 的分发，如果全部放进最内层循环，性能会很差。常见做法是把边界路径和主路径分开：主路径处理规则长度和对齐数据，尾部路径处理少量剩余元素。算子开发里，这种结构比在每次迭代里判断更清晰。

7.4 内存分配和生命周期

频繁分配释放内存会污染 benchmark，也会让推理延迟不稳定。推理引擎应尽量在模型加载或请求开始时准备缓冲区，热路径复用内存。张量对象要区分拥有数据和引用数据；临时缓冲区要有明确生命周期；跨线程共享缓冲区要明确所有权。

常见误区

不要在算子热路径里隐藏分配。一个看似普通的 `std::vector` 临时对象，可能在每次推理中分配内存，导致延迟抖动。教学代码可以简单，但 benchmark 前必须检查热路径分配。

7.5 优化顺序

本册建议的优化顺序是：先保证正确性，再记录测量边界；先减少大块无意义数据移动，再考虑 SIMD；先让单线程访问轨迹清晰，再加多线程；先检查算法和布局，再看微小指令级调整。这个顺序不保证每次最快，但能避免在错误层级上浪费时间。

习题与作业

检查 LCQI 推理路径中是否存在每次推理都会重新分配的 `vector`。把可以复用的临时缓冲区移动到执行器对象里，再比较推理延迟的平均值和波动。

第零部分

AI 基础：从张量到推理图

第 8 章

AI 模型、张量与推理图

本章从零建立推理需要的 AI 基础。我们不讨论训练算法的全部细节，而是聚焦推理引擎必须知道的概念：张量是什么，模型参数是什么，激活是什么，推理图怎样表达计算，算子接口如何连接数据。只有这些概念清楚，后面的 CPU 算子开发才不会变成“对着公式写循环”。

8.1 模型不是一个函数名

一个 AI 模型可以理解为一组参数和一张计算图。参数来自训练结果，例如权重矩阵、卷积核、归一化权重、Embedding 表。推理时输入数据进入计算图，经过一系列算子，产生输出。模型文件保存参数、结构和元数据；推理引擎负责把它加载到内存，并按正确顺序执行。

LCQI 第一阶段的模型很小：两层 MLP。它仍然具备完整模型的基本形态：第一层权重和 bias 是参数，输入向量是输入激活，隐藏层输出是中间激活，第二层输出是 logits。分类结果取 logits 最大值的下标。后续无论模型变成 CNN 还是 Transformer，本质仍然是参数加计算图。

8.2 张量、shape、stride 和 dtype

张量（tensor）是带形状的多维数组。shape 描述每一维长度，dtype 描述元素类型，stride 描述每一维移动一步时地址增加多少元素。连续行主序张量的 stride 可以从 shape 推出；切片、转置和某些布局转换会产生非连续 stride。推理引擎必须明确自己支持哪些布局，否则算子读错数据很难排查。

Listing 8.1 教学版张量视图：shape 和连续数据

```
1 #include <stdint>
2 #include <span>
3 #include <vector>
4
5 struct TensorView {
6     std::span<const float> data;
7     std::vector<std::int32_t> shape;
8 };
```

这个结构只表达只读 float 张量视图。真实引擎还要支持可写张量、不同 dtype、stride、内存所有权和设备位置。本册从简单结构开始，是为了让读者先看懂算子接口：算子接收张量，检查 shape，按约定布局读写数据。

8.3 参数和激活

参数是模型加载后长期存在的数据，推理过程中通常只读。激活是某次推理产生的中间数据，生命周期较短。这个区分影响内存管理和并发。参数可以被多个请求和线程共享；激活通常属于一次请求或一个 batch；临时缓冲区可以复用，但不能跨并发请求错误共享。

量化模型中，参数还包含 scale、zero point、分组大小等元数据。一个 int8 权重数组没有 scale 就没有真实数值意义。推理引擎的模型结构必须把量化数据和量化元数据绑在一起，不能只传裸指针。

8.4 推理图

推理图由节点和边组成。节点是算子，边是张量。线性层、ReLU、Softmax、卷积、归一化、Attention 都是节点。推理执行器按依赖顺序执行节点，管理每个边对应的内存。教学项目可以手写

固定顺序；更完整的引擎会解析模型文件，构建图，再做内存规划和算子调度。

心智模型

推理图是数据流，不是代码流。代码流描述程序怎样写，数据流描述张量怎样从一个算子变成下一个算子的输入。算子开发必须同时尊重这两条线。

8.5 正确性标准

推理引擎必须有正确性标准。对于 float 算子，常用绝对误差和相对误差；对于量化算子，要比较反量化结果或最终输出类别；对于随机输入，要固定种子；对于模型文件，要有格式校验。没有正确性测试的性能优化没有意义，因为它可能只是算错了。

习题与作业

把 LCQI 的两层 MLP 画成推理图，标出每条边的 shape 和 dtype。说明哪些数据是参数，哪些数据是激活，哪些数据是临时缓冲区。

第 9 章

常见 AI 算子与模型结构

理解 AI 算子之前，要先知道模型为什么由这些算子组成。推理引擎不是随意堆函数，而是在执行训练后确定下来的计算结构。不同模型结构使用不同算子，但许多底层模式相同：矩阵乘、逐元素变换、归一化、归约、查表、重排和数据搬运。

9.1 线性层和 MLP

线性层计算 $y = Wx + b$ 或批量形式 $Y = XW + b$ 。它把输入特征映射到新的特征空间，是 MLP、Transformer 投影层和分类头的基础。MLP 通常由线性层、激活函数和另一个线性层组成。LCQI 第一阶段选择 MLP，就是因为它用最小结构覆盖了参数、激活、矩阵运算和非线性。

激活函数让模型具有非线性表达能力。ReLU 是最简单的激活：小于零变成零，大于零保持不变。GELU、SiLU 等更常见于 Transformer，但计算更复杂。算子开发时，简单激活常常适合与前一个线性层输出融合，减少中间张量读写。

9.2 卷积网络

卷积网络常用于图像。卷积核在空间位置上滑动，提取局部模式。卷积算子涉及输入通道、输出通道、核大小、stride、padding 和布局。虽然卷积公式不同于矩阵乘，但实现上常常可以转化为 GEMM 或使用直接卷积、Winograd 等路径。读者理解卷积，是为了看到同一组硬件原则怎样作用在不同模型结构上。

9.3 Embedding 和查表

Embedding 把离散 token id 映射成向量。它本质上是查表：根据整数下标读取参数矩阵的一行。Embedding 的计算量不大，但内存访问可能不连续，容易受缓存和 TLB 影响。语言模型推理的第一步通常是 token id 查 Embedding，最后还可能用输出向量和词表矩阵计算 logits。

9.4 Transformer

Transformer 的核心部件包括多头 Attention、MLP、归一化、残差连接和位置编码。Attention 让每个 token 根据上下文聚合信息；MLP 对每个位置做特征变换；归一化稳定数值；残差连接帮助信息流动。推理时，prefill 阶段处理整段输入，decode 阶段逐 token 生成，并依赖 KV Cache 保存历史 key/value。

Transformer 看起来复杂，但落到底层仍然由算子组成：矩阵乘、Softmax、归一化、逐元素激活、查表、拼接和缓存读写。本册后续不会要求读者先成为模型训练专家，而是要能看懂这些算子在推理时怎样排列。

9.5 算子接口

一个算子接口至少要说明输入张量、输出张量、参数、shape 约束、dtype 约束和错误处理。算子内部可以有多种实现：标量版本用于正确性基线，SIMD 版本用于单核优化，多线程版本用于大规模输入，量化版本用于低带宽和低精度推理。接口稳定，内部实现才能逐步替换。

核心思想

AI 模型结构决定算子列表，算子接口决定推理引擎边界，CPU 性能工程决定每个算子在本地 Linux 上能跑多快。

习题与作业

选择一个简单网络，例如两层 MLP 或小型 CNN，把它拆成算子列表。对每个算子写出输入 shape、输出 shape、参数 shape 和主要计算模式。不要写代码，先把推理图讲清楚。

第零部分

算子开发：从数学到 CPU 内核

第 10 章

矩阵乘：从三重循环到性能结构

矩阵乘是 AI 计算里最重要的内核之一。全连接层、卷积降维、Transformer 的投影、MLP、Attention 中的 QK^T 和 PV 都可以直接或间接落到 GEMM。矩阵乘的公式很简单： $C_{ij} = \sum_k A_{ik} B_{kj}$ 。困难在于，这个公式对应的数据访问和复用空间极大，朴素三重循环通常离硬件上限很远。

10.1 先看最朴素的三重循环

假设矩阵按行主序存储， A 的形状是 $M \times K$ ， B 的形状是 $K \times N$ ， C 的形状是 $M \times N$ 。最直接的写法如下。

Listing 10.1 朴素 GEMM：按 i、j、k 顺序计算

```
1 #include <stdint>
2 #include <span>
3
4 void gemm_ijk(std::span<const float> a,
5               std::span<const float> b,
6               std::span<float> c,
7               std::int32_t m,
8               std::int32_t n,
9               std::int32_t k_size) {
10     for (std::int32_t i = 0; i < m; ++i) {
11         for (std::int32_t j = 0; j < n; ++j) {
12             float acc = 0.0F;
13             for (std::int32_t k = 0; k < k_size; ++k) {
14                 const float av =
15                     a[static_cast<std::size_t>(i) * k_size + k];
16                 const float bv =
17                     b[static_cast<std::size_t>(k) * n + j];
18                 acc += av * bv;
19             }
20             c[static_cast<std::size_t>(i) * n + j] = acc;
21         }
22     }
23 }
```

这个版本对 A 的访问是顺序的，对 B 的访问是跨行跳跃的：固定 j ， k 每增加一，地址增加 N 个 float。如果 N 很大，每次访问 B 都可能落在新的 cache line 上。数学上， B 的同一行会被多个 j 使用， A 的同一个元素会被多个输出列使用，但这个循环顺序并没有很好地让缓存捕获这些复用。

10.2 循环顺序改变复用位置

把循环顺序改成 i, k, j ，可以让 B 的一整行被顺序扫描，同时让 A_{ik} 复用于多个输出列。代码如下。

Listing 10.2 按 i、k、j 顺序暴露 A 元素对一行 C 的复用

```
1 #include <algorithm>
2 #include <stdint>
```

```

3 #include <span>
4
5 void gemm_ikj(std::span<const float> a,
6             std::span<const float> b,
7             std::span<float> c,
8             std::int32_t m,
9             std::int32_t n,
10            std::int32_t k_size) {
11     std::fill(c.begin(), c.end(), 0.0F);
12     for (std::int32_t i = 0; i < m; ++i) {
13         for (std::int32_t k = 0; k < k_size; ++k) {
14             const float av =
15                 a[static_cast<std::size_t>(i) * k_size + k];
16             for (std::int32_t j = 0; j < n; ++j) {
17                 c[static_cast<std::size_t>(i) * n + j] +=
18                     av * b[static_cast<std::size_t>(k) * n + j];
19             }
20         }
21     }
22 }

```

这个版本常常比 i, j, k 顺序更友好，因为内层 j 顺序访问 B 和 C 。但它也带来新问题： C 的同一行会被反复读写 K 次。如果 N 很大， C 行不能稳定留在高层缓存里，读写流量仍然很高。矩阵乘优化不是找到一个永远最好的循环顺序，而是让 A 、 B 、 C 的工作块同时适合缓存和寄存器。

10.3 复杂度相同，常数完全不同

两个版本的时间复杂度都是 $O(MKN)$ ，但性能可能差很多。这说明大 O 只能描述增长阶，不能描述内存层级、SIMD 和常数因素。对于 AI 内核，复杂度分析仍然有用，因为它告诉我们计算量规模；但要预测真实性能，还必须把地址序列和复用结构写出来。

矩阵乘有一个重要性质：每个 A_{ik} 会参与 N 个输出，每个 B_{kj} 会参与 M 个输出。如果能把一小块 A 和一小块 B 留在缓存或寄存器里，它们就能产生一小块 C 的许多结果。这就是分块的基本动机。后面的章节会看到，工业 GEMM 把这个思想分成多层：L3 cache 级别的大块、L2 cache 级别的 panel、L1 cache 级别的 packed block、寄存器级别的 micro tile。

核心思想

矩阵乘快，不是因为公式神秘，而是因为它有高复用潜力。优化的核心是让这种潜力真实落到缓存、寄存器和 SIMD lane 上，而不是只停留在数学表达式里。

10.4 误差和累加顺序

浮点矩阵乘还涉及数值问题。不同循环顺序、不同分块、不同 SIMD 归约顺序，可能得到略有差异的结果，因为浮点加法不满足严格结合律。高性能库通常接受这种微小差异，并用误差容忍检查正确性。教材实验中不要用完全相等比较 float，而应使用绝对误差和相对误差。

Listing 10.3 浮点结果检查：同时看绝对误差和相对误差

```

1 #include <cmath>
2
3 bool close_enough(float actual, float expected) {
4     const float abs_error = std::fabs(actual - expected);
5     const float scale = std::max(1.0F, std::fabs(expected));
6     return abs_error <= 1.0e-4F * scale;
7 }

```

习题与作业

实现 i, j, k 和 i, k, j 两个版本，使用相同输入验证结果接近。随后固定 M 和 K ，逐步增加 N ，观察两个版本的差距如何变化。把观察结果和 B 、 C 的访问轨迹联系起来解释。

第 11 章

分块、Packing 与微内核

上一章看到，改变循环顺序可以改善一部分访问模式，但还不够。真正高性能的 GEMM 往往会分成多层结构：先把矩阵切成块，再把块复制成更适合内核访问的 packed 格式，最后用一个手写或生成的微内核在寄存器里计算小块 C 。这一章解释这些结构为什么存在。

11.1 分块的根本目的

分块的目标是控制工作集大小。假设我们计算 C 的一个小块，它需要 A 的若干行和 B 的若干列。只要这部分 A 、 B 、 C 能留在合适缓存层级里，内层计算就能反复复用它们。若块太大，缓存装不下，数据被提前挤出；若块太小，循环开销、边界处理和 packing 成本占比过高。

可以先写一个教学版分块，不考虑最优块大小。

Listing 11.1 教学版分块 GEMM：按块组织复用

```
1 #include <algorithm>
2 #include <cstdint>
3 #include <span>
4
5 void gemm_blocked(std::span<const float> a,
6                  std::span<const float> b,
7                  std::span<float> c,
8                  std::int32_t m,
9                  std::int32_t n,
10                 std::int32_t k_size,
11                 std::int32_t block) {
12     std::fill(c.begin(), c.end(), 0.0F);
13     for (std::int32_t ii = 0; ii < m; ii += block) {
14         for (std::int32_t kk = 0; kk < k_size; kk += block) {
15             for (std::int32_t jj = 0; jj < n; jj += block) {
16                 const std::int32_t i_end = std::min(ii + block, m);
17                 const std::int32_t k_end = std::min(kk + block, k_size);
18                 const std::int32_t j_end = std::min(jj + block, n);
19                 for (std::int32_t i = ii; i < i_end; ++i) {
20                     for (std::int32_t k = kk; k < k_end; ++k) {
21                         const float av =
22                             a[static_cast<std::size_t>(i) * k_size + k];
23                         for (std::int32_t j = jj; j < j_end; ++j) {
24                             c[static_cast<std::size_t>(i) * n + j] +=
25                                 av * b[static_cast<std::size_t>(k) * n + j];
26                         }
27                     }
28                 }
29             }
30         }
31     }
32 }
```

这段代码还不是工业实现，但它已经表达了分块思想。它不是减少总 FLOPs，而是改变数据在

缓存里的停留方式。分块参数需要按目标机器调优，因为 L1、L2、L3 容量、cache associativity、SIMD 宽度、寄存器数量和预取行为都会影响最佳值。

11.2 为什么需要 Packing

分块之后，仍然可能存在跨步访问和边界复杂性。Packing 的做法是把即将计算的一块 A 或 B 复制到连续、对齐、顺序访问的临时缓冲区。初看这像额外成本：明明原矩阵里已经有数据，为什么还要复制一次？答案是复制一次可以换来内层微内核大量规则访问。只要 packed block 被复用足够多次，复制成本就能被摊薄。

Packing 还让微内核更简单。微内核不必处理原矩阵的 leading dimension、跨步、边界和奇怪布局，只要按固定格式读取连续数据。许多库会把 B 的 panel 打包成适合连续向量加载的形状，把 A 的小块打包成适合广播或向量加载的形状。这样内层循环就能专注于 FMA 和寄存器调度。

心智模型

Packing 是一次主动的数据重排。它用可控的顺序拷贝，换取后续大量计算中的规则加载、少分支、少地址计算和更稳定的预取效果。

11.3 微内核和寄存器块

微内核通常计算 C 的一个很小 tile，例如 $M_R \times N_R$ 。这个 tile 的累加值保存在寄存器中，循环遍历 K 维，每一步从 A 和 B 取数据并执行 FMA。等 K 维完成后，再把寄存器里的 tile 写回 C 。

为什么要把 C 的小块放在寄存器里？因为 C 的每个元素要累加很多次。如果每次乘加都读写内存，流量巨大；若把部分和保存在寄存器，内层只在最后写回一次。寄存器数量决定了 tile 不能无限大，SIMD 宽度决定了 N_R 或 M_R 常常按向量 lane 组织。

Listing 11.2 教学化微内核形状：寄存器保存 2x4 的 C 小块

```

1 #include <stdint>
2 #include <span>
3
4 void microkernel_2x4(std::span<const float> a,
5                     std::span<const float> b,
6                     std::span<float> c,
7                     std::int32_t k_size,
8                     std::int32_t c_stride) {
9     float c00 = 0.0F;
10    float c01 = 0.0F;
11    float c02 = 0.0F;
12    float c03 = 0.0F;
13    float c10 = 0.0F;
14    float c11 = 0.0F;
15    float c12 = 0.0F;
16    float c13 = 0.0F;
17
18    for (std::int32_t k = 0; k < k_size; ++k) {
19        const float a0 = a[static_cast<std::size_t>(k)];
20        const float a1 = a[static_cast<std::size_t>(k_size) + k];
21        const std::size_t b_base = static_cast<std::size_t>(k) * 4U;
22        const float b0 = b[b_base + 0U];
23        const float b1 = b[b_base + 1U];
24        const float b2 = b[b_base + 2U];
25        const float b3 = b[b_base + 3U];
26        c00 += a0 * b0;
27        c01 += a0 * b1;

```

```

28     c02 += a0 * b2;
29     c03 += a0 * b3;
30     c10 += a1 * b0;
31     c11 += a1 * b1;
32     c12 += a1 * b2;
33     c13 += a1 * b3;
34 }
35
36 c[0] = c00;
37 c[1] = c01;
38 c[2] = c02;
39 c[3] = c03;
40 c[static_cast<std::size_t>(c_stride) + 0U] = c10;
41 c[static_cast<std::size_t>(c_stride) + 1U] = c11;
42 c[static_cast<std::size_t>(c_stride) + 2U] = c12;
43 c[static_cast<std::size_t>(c_stride) + 3U] = c13;
44 }

```

这段代码没有使用 intrinsic，但已经体现微内核思想。真正的 SIMD 版本会把一行或多行 B 装进向量寄存器，把 A 的标量广播到向量，执行向量 FMA。寄存器块越大，复用越充分，但寄存器压力也越大。超过寄存器容量后，编译器会 spill 到栈上，性能反而下降。

11.4 工业库的分层视角

OpenBLAS、BLIS、oneDNN 和其他库的 GEMM 结构看起来复杂，是因为它们要同时服务不同矩阵形状、不同数据类型、不同指令集和不同缓存层级。常见结构可以粗略理解为：外层按大块切分，适配 L3 和多线程；中层 packing panel，适配 L2；内层微内核，适配 L1、寄存器和 SIMD。源码阅读时不要一上来陷入所有宏和平台分支，先找出这三层结构。

习题与作业

阅读一个开源 BLAS 的 GEMM 路径，记录它在哪里选择 block size，在哪里 packing，在哪里调用架构相关 microkernel。不要求读懂所有汇编，先把数据从原矩阵到 packed buffer 再到 microkernel 的路径画出来。

第 12 章

卷积、归一化与 Softmax 算子

矩阵乘是算子开发的核心，但不是全部。推理引擎还需要卷积、归一化、Softmax、激活函数和残差加法等算子。它们的共同特点是：公式通常不难，性能和正确性细节很多。卷积考验数据布局 and 复用，归一化考验归约和内存流量，Softmax 考验数值稳定性。

12.1 卷积的数据流

卷积在视觉模型中常见。直接卷积会遍历 batch、输出通道、输出空间、输入通道和卷积核窗口。输入像素被相邻窗口复用，权重被多个输出位置复用。优化卷积时，必须决定数据布局、循环顺序和是否把卷积转化为 GEMM。

im2col 方法把每个卷积窗口展开成矩阵，再调用 GEMM。它用额外内存换取规则矩阵乘。直接卷积减少中间数据，但访问更复杂。Winograd 可以减少小卷积核乘法数量，但增加变换和数值误差。工程上没有永远最好的卷积算法，只有适合当前 shape、dtype 和硬件的选择。

12.2 归一化

LayerNorm 和 RMSNorm 是 Transformer 中常见算子。它们先对一段向量做归约，得到均值、方差或平方均值，再对每个元素缩放。归一化的 FLOPs 不一定高，但会扫描完整向量，常常受内存带宽和归约结构影响。

Listing 12.1 RMSNorm 的教学实现

```
1 #include <cmath>
2 #include <cstdint>
3 #include <span>
4
5 void rms_norm(std::span<const float> x,
6               std::span<const float> weight,
7               std::span<float> y,
8               float epsilon) {
9     float square_sum = 0.0F;
10    for (std::int64_t i = 0; i < static_cast<std::int64_t>(x.size()); ++i) {
11        const float value = x[static_cast<std::size_t>(i)];
12        square_sum += value * value;
13    }
14    const float mean_square = square_sum / static_cast<float>(x.size());
15    const float scale = 1.0F / std::sqrt(mean_square + epsilon);
16    for (std::int64_t i = 0; i < static_cast<std::int64_t>(x.size()); ++i) {
17        y[static_cast<std::size_t>(i)] =
18            x[static_cast<std::size_t>(i)] * scale *
19            weight[static_cast<std::size_t>(i)];
20    }
21 }
```

优化归一化时，要考虑多累加器、SIMD 归约、epsilon、低精度累加和融合。不要为了少一次扫描破坏数值稳定性。

12.3 Softmax

Softmax 把一组 logits 转成概率分布。稳定实现要先减最大值，再计算指数和归一化。直接计算 $\exp(x_i)$ 可能溢出；减去最大值不会改变概率比例，却能显著改善数值稳定性。

Listing 12.2 稳定 Softmax 的基本结构

```

1 void softmax_stable(std::span<const float> input,
2                     std::span<float> output) {
3     float max_value = input[0];
4     for (std::int64_t i = 1; i < static_cast<std::int64_t>(input.size()); ++i) {
5         max_value = std::max(max_value, input[static_cast<std::size_t>(i)]);
6     }
7     float sum = 0.0F;
8     for (std::int64_t i = 0; i < static_cast<std::int64_t>(input.size()); ++i) {
9         const float value =
10             std::exp(input[static_cast<std::size_t>(i)] - max_value);
11         output[static_cast<std::size_t>(i)] = value;
12         sum += value;
13     }
14     const float inv_sum = 1.0F / sum;
15     for (std::int64_t i = 0; i < static_cast<std::int64_t>(output.size()); ++i) {
16         output[static_cast<std::size_t>(i)] *= inv_sum;
17     }
18 }

```

12.4 和 LCQI 的关系

LCQI 第一阶段不必实现卷积和完整 Attention，但要实现 ReLU、线性层和输出分类。后续扩展到 CNN 时加入卷积，扩展到语言模型时加入归一化、Softmax、RoPE 和 Attention。每加入一个算子，都要先写标量正确版本，再写测试，再考虑 SIMD、多线程和量化。

常见误区

算子开发不要直接从最快版本开始。没有标量基线和误差检查，后续 SIMD 或多线程版本很难证明自己正确。

第零部分

量化推理引擎：从算子到本地运行

第 13 章

Attention、KV Cache 与内存流量

Attention 是 Transformer 的核心。推理时，它不仅包含矩阵乘，还包含 Softmax、mask、KV Cache 读写和随序列长度增长的内存访问。理解 Attention 性能，必须区分 prefill 和 decode：prefill 处理整段输入，矩阵形状较大；decode 每次生成一个 token，batch 和查询长度通常很小，但要读取历史 KV Cache。

13.1 Attention 的计算图

单头 Attention 可以写成 $O = \text{softmax}(QK^T/\sqrt{d})V$ 。多头 Attention 只是把 hidden dimension 切成多个 head 并并行计算。这个公式里有两个主要矩阵乘： QK^T 产生注意力分数，Softmax 归一化后再乘 V 得到输出。若序列长度为 S ，head dimension 为 D ，那么分数矩阵大小是 $S \times S$ ，长序列时中间矩阵非常大。

prefill 阶段， Q 、 K 、 V 都来自当前输入序列，计算类似批量矩阵乘。decode 阶段，新 token 只有一个或少量 query，但必须与历史所有 key 做点积，再对所有历史位置 Softmax，然后加权所有 value。此时计算量随历史长度线性增长，KV Cache 的读取成为关键。

13.2 KV Cache 的作用

如果每生成一个 token 都重新计算所有历史 token 的 K 和 V ，成本会非常高。KV Cache 把历史 token 的 key 和 value 保存下来，新 token 到来时只计算它自己的 Q, K, V ，然后用新 Q 与缓存中的所有历史 K 做 Attention。这样减少了重复计算，但增加了持续增长的内存占用和读取流量。

KV Cache 的布局直接影响性能。常见维度包括 layer、batch、head、sequence、head dimension。哪一维连续，决定 decode 时读取一个 head 的历史 key 是否顺序，多个 head 是否容易并行，追加新 token 是否连续写入。布局还影响量化 KV Cache、分页 KV Cache 和多请求 batching。

Listing 13.1 教学版 KV Cache 索引：明确每个维度的地址贡献

```
1 #include <cstdint>
2 #include <cstdint>
3
4 struct KvShape {
5     std::int32_t layers;
6     std::int32_t batch;
7     std::int32_t heads;
8     std::int32_t max_seq;
9     std::int32_t head_dim;
10 };
11
12 std::size_t kv_index(const KvShape& shape,
13                     std::int32_t layer,
14                     std::int32_t batch_id,
15                     std::int32_t head,
16                     std::int32_t token,
17                     std::int32_t dim) {
18     std::size_t index = static_cast<std::size_t>(layer);
19     index = index * shape.batch + static_cast<std::size_t>(batch_id);
20     index = index * shape.heads + static_cast<std::size_t>(head);
21     index = index * shape.max_seq + static_cast<std::size_t>(token);
```

```

22     index = index * shape.head_dim + static_cast<std::size_t>(dim);
23     return index;
24 }

```

这个布局让同一个 token 的 head dimension 连续。如果 decode 时要对固定 head 扫描所有历史 token，并且每个 token 读取完整 head dimension，那么 token 维和 dim 维的组合访问是规则的。但如果 batching、分页或量化改变布局，就要重新分析地址序列。

13.3 从完整矩阵到在线 Softmax

朴素 Attention 会先算完整分数矩阵，再做 Softmax，再乘 V 。这对长序列很昂贵，因为分数矩阵写入和读出都很大。在线 Softmax 的思想是按块处理分数，维护当前最大值和归一化分母。当新块到来且最大值变化时，旧的累加结果按比例缩放。这样可以不把完整分数矩阵写回内存。

这类算法说明，Attention 优化不是简单“调用更快 GEMM”。它要把矩阵乘、Softmax 和乘 V 融合成数据流问题。分块大小要适配缓存，统计量要保持数值稳定，mask 和 causal 约束要正确处理，尾部和不同序列长度也要覆盖。

13.4 decode 阶段的瓶颈

decode 阶段常见瓶颈是内存带宽和小矩阵效率。每生成一个 token，都要读取每层的 KV Cache。随着上下文变长，读取历史 KV 的字节数增长，而 query 很少，矩阵乘形状偏瘦，难以达到大 GEMM 的峰值效率。此时优化方向包括 KV Cache 量化、分组查询注意力、分页管理、连续 batching、减少中间写回和更好的线程切分。

常见误区

不要用 prefill 的吞吐推断 decode 的速度。prefill 更像大矩阵计算，decode 更像长历史缓存扫描加小矩阵计算。两者的瓶颈、批处理策略和优化手段都不同。

13.5 源码阅读入口

阅读 llama.cpp、vLLM、oneDNN Graph 或 XNNPACK 这类项目时，Attention 路径可以按四个问题定位。第一， Q, K, V 投影使用什么矩阵乘路径。第二，KV Cache 的物理布局是什么，追加和读取如何索引。第三，Softmax 是否与分数计算或乘 V 融合。第四，prefill 和 decode 是否走不同 kernel。只要这四个问题清楚，复杂代码就能分层理解。

习题与作业

给定 batch 为 1、head 数为 32、head dimension 为 128、上下文长度为 4096 的 decode 场景，估算一层读取 K 和 V 至少需要多少字节。再乘以层数，观察为什么 KV Cache 带宽会成为长上下文推理的重要问题。

第 14 章

本地量化推理引擎

本章把全书的线索收束到贯穿项目：Linux CPU Quantized Inference。我们要在本地 Linux 上加载一个小型量化模型，执行推理，验证输出，并记录性能。这个项目规模不大，但它包含推理引擎的核心骨架，后续可以继续扩展到卷积、Attention 和更复杂模型格式。

14.1 模型文件

教学模型文件采用文本格式，便于手工检查。文件记录输入维度、隐藏维度、输出维度、每层 scale、int8 权重和 float bias。真实工业模型常用二进制格式，是为了加载速度、体积和元数据完整性；教学阶段先用文本格式，是为了让读者看清每个数字进入推理路径的位置。

14.2 量化线性层

第一阶段使用 int8 权重和 float 激活。计算时把权重按 scale 反量化，再与输入相乘累加。这个实现不是最高效的 int8 GEMM，但语义清楚，适合作为正确性基线。后续可以把反量化和累加融合，或使用 int32 累加和输出阶段 scale。

Listing 14.1 量化线性层的第一阶段语义

```
1 float weighted_sum_i8(const std::int8_t* weights,
2                       const float* input,
3                       float scale,
4                       std::int32_t input_size) {
5     float acc = 0.0F;
6     for (std::int32_t i = 0; i < input_size; ++i) {
7         acc += static_cast<float>(weights[i]) * scale * input[i];
8     }
9     return acc;
10 }
```

14.3 推理执行

推理执行器先运行隐藏层，写入隐藏激活并执行 ReLU；再运行输出层，得到 logits；最后取最大 logit 的下标作为分类结果。这个流程简单，但已经有模型参数、激活缓冲区、算子调度和输出解释。测试用固定输入和模型文件验证结果，benchmark 重复执行多次记录平均延迟。

14.4 正确性和性能报告

正确性报告至少包含模型文件、输入、期望输出、实际输出和误差。性能报告至少包含 CPU 信息、编译器、编译选项、线程数、重复次数、平均延迟和吞吐。若开启多线程，还要记录任务切分方式。若后续引入 SIMD 或量化累加优化，要保留标量基线对比。

核心思想

推理引擎不是一个巨大函数，而是一组边界清晰的模块：模型加载、张量表示、量化权重、算子内核、执行器、测试和 benchmark。模块边界清楚，后续优化才不会破坏正确性。

14.5 后续扩展

LCQI 的自然扩展路径有四条。第一，支持更多算子，例如卷积、RMSNorm、Softmax 和 Attention。第二，改进量化格式，例如 per-channel scale、group-wise quantization 和 int32 累加。第三，加入线程池和更稳定的任务切分。第四，支持更接近真实模型的二进制模型格式。每次扩展都必须保留测试和 benchmark。

本章实验：构建 LCQI，运行测试和 benchmark。确认模型文件可以在本地 Linux 上加载，推理输出分类下标正确，并记录单次推理平均耗时。

附录 A

实验路线

第二册的实验围绕 Linux CPU Quantized Inference 贯穿项目展开。第一组实验是工程骨架，包括模型文件加载、张量结构、量化线性层、ReLU、argmax、CLI 推理和 CTest 正确性测试。第二组实验是测量基础，包括向量加法、点积、矩阵乘三种内核的时间、FLOPs、字节数和 perf 计数器。第三组实验是内存层级，包括行访问和列访问、AoS 和 SoA、不同矩阵规模下的缓存行为。

第四组实验是并发和同步，包括线程池、任务粒度、锁、原子操作、false sharing 和 NUMA 观察。第五组实验是 SIMD，包括自动向量化报告、点积多累加器、尾部处理和汇编检查。第六组实验是 GEMM，从 i, j, k 朴素版本、 i, k, j 版本、分块版本、packing 版本逐步推进，再对比 OpenBLAS 或 oneDNN 的性能。第七组实验是 AI 算子，包括稳定 Softmax、RMSNorm、RoPE、Attention 的 prefill 和 decode。第八组实验是量化推理，包括 int8 对称量化、分组 scale、误差统计、量化矩阵乘和 LCQI 端到端 benchmark。

每个实验必须记录环境、输入、命令、输出、正确性检查和结论。性能数据没有上下文就不能进入正文。若实验受手机、Termux 或虚拟化环境限制，也要明确记录限制。

附录 B

源码阅读索引

本附录保存后续源码专题的入口。

OpenBLAS 和 BLIS 用于阅读 GEMM 的经典分层结构：block size、packing、microkernel、架构分发和多线程切分。阅读时优先找 SGEMM 路径，不要一开始陷入所有数据类型。

oneDNN 用于阅读工业深度学习后端：memory descriptor、primitive、post-op 融合、ISA 选择、卷积和矩阵乘实现。阅读时重点观察 layout 和 post-op 如何影响内核选择。

XNNPACK 用于阅读移动端和 CPU 推理内核：operator 创建、microkernel 表、indirection buffer、threadpool 和量化路径。它适合理解小算子和低延迟场景。

llama.cpp 和 ggml 用于阅读大语言模型 CPU 推理：tensor 表示、计算图、后端分发、量化 block、KV Cache、RMSNorm、RoPE、Attention 和采样。阅读时应区分模型加载、图构建、单 token decode 和 benchmark 路径。

Linux 内核源码用于理解调度、NUMA、页表、透明大页、perf 事件和内存分配。第二册不会把 Linux 内核作为主线，但性能工程遇到线程绑定、页错误、NUMA 和 perf 计数器时，需要知道这些机制来自哪里。

术语表

算术强度 操作数与数据移动字节数的比值，用来判断一个内核更可能受计算吞吐还是内存带宽限制。

Roofline 用计算峰值、内存带宽和算术强度估算性能上界的模型。

SIMD 一条指令处理多个数据 lane 的执行方式，是现代 CPU 单核数值吞吐的重要来源。

分块 把大问题切成适合缓存或寄存器的小块，以提高数据复用。

Packing 把矩阵块复制成连续、对齐、适合微内核访问的临时布局。

微内核 GEMM 或卷积中最内层的架构相关计算核心，通常直接围绕寄存器和 SIMD 指令设计。

KV Cache Transformer decode 阶段保存历史 key 和 value 的缓存，用于避免重复计算历史 token。

量化 用低位宽整数或低精度浮点表示权重、激活或缓存，并通过 scale、zero point 等元数据近似原始实数。

NUMA 非统一内存访问结构，不同核心访问不同内存节点的成本不同。

算子融合 把多个相邻算子合并执行，减少中间张量读写和调度开销。

索引

AoS, 7

SoA, 7

张量, 19

计算强度, 8